

Skill Progressive Disclosure

两个问题

一个 Agent 可以安装很多 Skill，但一次任务通常只需要其中少数几个。若启动时把所有操作步骤、参考资料都放进上下文，则每个任务都会携带大量无关内容。Skill 渐进式披露（Progressive Disclosure）的作用，是先让模型知道有哪些能力，再按任务需要加载具体内容。

这里有两个问题：Skill 如何分层进入上下文？模型只读了 description，为什么知道还要继续读取 SKILL.md？

整体过程是：发现技能 → 提供目录与加载规则 → 判断任务是否匹配 → 调用工具 → 正文进入上下文。Pi 和 DeepSeek Harness 都遵循这个过程，但目录的位置、路径的暴露方式和加载工具不同。

Pi：通过 read 读取 Skill

发现技能，构造目录

首先由宿主程序扫描技能目录，例如项目中的 .agents/skills/。具体搜索路径还取决于 Harness 的配置，不能把这个示例当成唯一入口。

发现 Skill 后，Pi 将 name、description、location 组织为目录，加入 system prompt。三者分别回答：技能叫什么、适用于什么任务、正文在哪里。

下面只展示目录结构，内容为示意：

```
<available_skills>
  <skill>
    <name>translate</name>
    <description> 翻译文本，并遵循项目术语与语言规范</description>
    <location>/project/.agents/skills/translate/SKILL.md</location>
  </skill>
</available_skills>
```

此时模型能看见技能的用途和位置，但还没有拿到它的具体翻译流程。description 的作用是帮助选择；操作步骤保留在正文中。

这里的“先加载元信息”指先把元信息放入模型上下文。宿主为了解析 frontmatter，可能已经从磁盘读取了整个文件。Pi 的发现逻辑就会读取全文，再提取目录字段。宿主读过文件，与模型看见全文，是两件事。

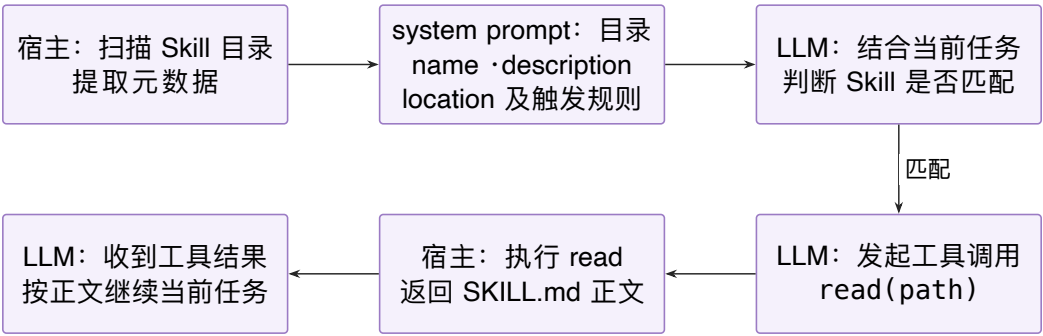
加载规则把 description 与工具连接起来

只有技能目录，还没有说明接下来如何使用。Pi 同时提供明确的规则：当任务匹配某个 Skill 的描述时，使用 read 工具读取它的文件。

这条规则包含三个部分：触发条件是任务匹配 description；动作是调用 read；读取目标是 location 指向的文件。因此，模型继续读正文所需的信息，是“目录 + 触发规则 + 可用工具”共同提供的。

具体调用可以写成以下结构示意图：

```
LLM 判断任务需要 translate
-> read({ path: "/project/.agents/skills/translate/SKILL.md" })
宿主执行读取
-> 将文件内容作为工具结果加入上下文
LLM 结合正文，决定下一步操作
```



SKILL.md 正文在 read 的工具结果返回后，才进入模型上下文。

Pi：目录和规则先进入上下文，正文通过工具结果返回

模型负责判断相关性并生成工具调用，宿主负责实际读取文件、回传结果。description 本身不会执行加载；文件放在硬盘上，也不会自动成为模型的输入。

这种自动选择仍可能漏选或选错。描述需要写清适用任务和触发条件，让模型能够区分相邻能力。可以分别用应当触发、不应触发和容易混淆的请求检查描述，再调整边界。

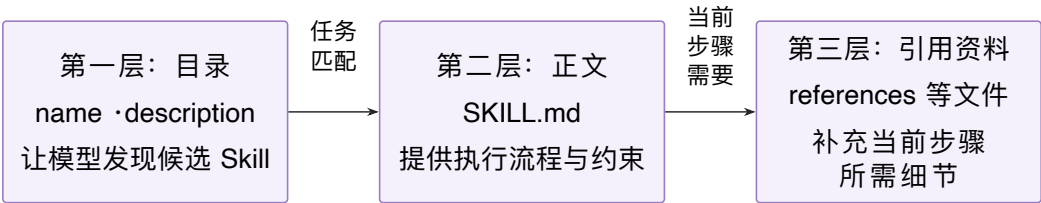
三层内容，三种加载时机

第一层是技能目录，主要提供名称与描述。模型先知道有哪些候选能力，判断当前任务是否需要它们。

第二层是 SKILL.md 正文，提供步骤、约束和完成标准。任务匹配后再加载，模型才获得如何执行这类任务的具体指示。

第三层是正文引用的资料，例如 references/ 中的详细规范、模板说明或补充指南。模型在执行到相关步骤、需要这些细节时再读取。

渐进式披露：控制信息进入模型上下文的时机



层级指进入模型上下文的时机；宿主在扫描元数据时，仍可能读取完整文件。

渐进式披露：目录、正文、引用资料分别按需进入上下文

例如正文引用了 references/translate-guide.md, 读取 SKILL.md 并不等于这份指南也已读入。需要先依据 Skill 所在目录解析相对路径，再由相应工具取得内容。正文应写清什么时候需要这份指南，否则只是把资料放到了另一个文件里。

因此，渐进式披露需要同时设计内容边界与进入下一层的条件。目录负责选择，正文负责执行，补充资料负责提供当前步骤所需的细节。是否继续加载，取决于任务需要。

分层也需要保证必要信息能够到达模型。使用该 Skill 时都必须遵守的核心约束，应留在正文；只适用于某个分支的细节，再放入引用资料，并写明读取条件。若关键限制只藏在参考文件里，而正文没有给出触发条件，模型可能在读到限制之前就已采取行动。

DeepSeek Harness：通过 skill 按名称加载

目录作为 user-role 消息进入 context

DeepSeek Harness (DSH) 同样先发现技能，但模型可见的目录只包含 name 与 description。目录由宿主生成，以 user-role 消息进入会话上下文。

这条消息不是用户手动补发的请求，而是 Harness 为模型组织的技能提示。目录旁仍有明确规则：若用户点名某个 Skill，或任务清楚匹配其描述，则先调用 skill 工具加载完整指示，再执行任务。

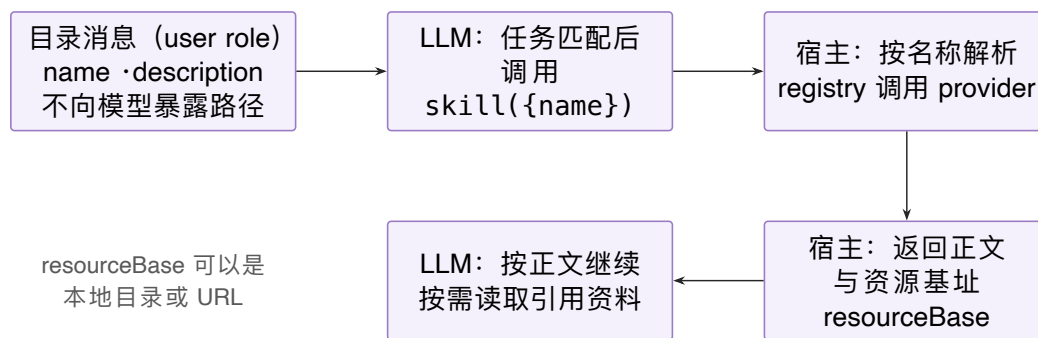
system prompt 是 context 的一部分；user-role 消息也属于 context。这里比较的是目录以什么角色进入模型输入。消息正文即使使用 <system-reminder> 标签，实际角色也仍由消息的 role 决定。

名称交给工具，路径由宿主解析

Pi 使用通用文件读取接口，因此需要告诉模型文件路径。DSH 提供专门的 skill 工具，模型只需传入目录中的准确名称：

```
skill({ name: "translate" })
```

宿主收到调用后，根据技能名称查找注册项，再由对应 provider 加载正文。名称如何对应到本地文件或其他资源，由宿主处理，初始目录不必把路径交给模型。



路径与来源由宿主管理，模型以 Skill 名称发起加载。

DSH：模型按名称请求，宿主解析来源并返回正文

加载结果需要让模型获得具体指示，以及继续访问引用资料所需的位置信息。在本文采用的 DSH 实现中，工具执行结果包含正文 content，并可附带资源基址 resourceBase；宿主将其组织成模型可读的正文与资源提示。对于文件系统中的 Skill，这个基址就是技能目录。

所以，“目录没有 location”只意味着路径不在初始候选列表中。加载正文后，模型仍可能需要资源基址，才能正确解析正文中的相对引用。模型不再需要在加载请求中提供文件路径，名称到资源的解析交给宿主。

skill({name}) 是一次加载请求。宿主执行成功后，将正文与资源提示回传，下一轮模型才能依据这些内容继续工作。这个过程不会自动读入整个 Skill 文件夹或执行其中所有脚本；正文进入上下文，也不意味着它变成了一条 system message。

两种实现的区别

对比项	Pi 的 read 路径	DSH 的 skill 路径
目录进入位置	system prompt	user-role 会话消息
模型可见的目录字段	name、description、location	name、description
模型发起的加载请求	read(path)	skill(name)
定位正文所需的信息	目录给出文件路径	宿主按名称查找来源

表中比较的是 Pi 的通用 read 加载方式与 DSH 的专用 skill 加载方式。Pi 代码固定于 ce950d78，DSH 固定于 ddefc45f；字段与接口以这些实现为准。

目录放在哪里，与工具如何定位正文，是两个设计维度。user-role 消息同样可以携带路径，system prompt 也可以要求模型调用按名称加载的工具。Pi 暴露 location、DSH 在目录中省略路径，直接对应的是两者加载接口所需的参数。

通用 read 复用已有的文件读取能力，过程直接；专用 skill 将按名称查找、来源解析和结果组织集中在一个接口中。前者需要模型在调用中给出路径，后者把名称到来源的映射交给宿主。选择哪一种，取决于 Harness 如何组织工具和资源。

自动选择与显式加载

前面的流程依赖模型判断任务是否匹配描述。宿主还可以提供显式加载入口，让用户直接选定 Skill。

在本文采用的 Pi 实现中，输入 /skill:translate 后，宿主直接读取正文并展开到用户消息中，无需先让模型决定是否调用 read。DSH 也支持对允许用户调用的技能使用 /translate 这样的指令，由宿主加载正文。

自然语言“请使用 translate”与宿主识别的加载指令，经过的路径可能不同。前者可以触发模型调用工具，后者可以直接由宿主完成加载。无论采用哪条路径，读取引用资料、运行脚本仍需要后续工具调用，能否执行也取决于工具与环境权限。

目录成本与更新边界

渐进式披露减少了无关正文的预先加载，目录本身仍占用上下文。模型可见的 Skill 越多、描述越长，候选列表就越大。描述应足以区分任务，具体操作步骤留给正文；第三层资料则只在相关步骤需要时读取。

内容更新还要区分目录与正文。在本文采用的 DSH 实现中，后续模型步骤会检查目录；模型可见的名称、描述或成员变化时，向会话追加完整的替换目录，指示以新目录为准。旧目录仍保留在历史中。

仅修改正文，目录可以保持不变。该实现再次通过 skill 加载时会读取当前正文，但此前返回的旧正文不会自动改写。因此，“文件已更新”与“当前上下文已包含新正文”是两个状态。需要应用新规则时，应明确重新加载，让后续步骤知道采用哪个版本；已经执行的操作则需要另外检查。

总结

(1) Skill 渐进式披露控制的是内容进入模型上下文的时机：先目录，再正文，最后按需读取引用资料。宿主读取文件与模型获得内容，需要分开理解。

(2) 模型从描述走向正文，需要明确的加载规则和可执行的工具接口；用户也可以通过显式入口选定 Skill。description 提供选择依据，加载后的正文再提供具体指示。

(3) Pi 与 DSH 展示了两种实现：按路径使用通用 read，或按名称使用专用 skill。共同的设计要求是让每层内容边界清楚，并写明进入下一层的条件。